Java Collection Framework and Strings

Student-friendly revision guide • 50 questions • Viva-ready answers

A. Java Collection Framework

1. What is the Java Collection Framework?

Answer: A set of interfaces, classes, and algorithms for storing and working with groups of objects. Core interfaces include **List**, **Set**, **Queue**, and **Map**.

2. Why do we use collections instead of arrays?

Answer: Collections can grow or shrink dynamically and provide ready-made operations such as searching, sorting, iteration, and duplicate handling. Arrays have a fixed size.

3. What is the difference between Collection and Collections?

Answer: Collection is the root interface for groups of objects. **Collections** is a utility class with static methods such as `sort()`, `reverse()`, and `max()`.

4. What is the difference between Collection and Map?

Answer: A **Collection** stores individual elements. A **Map** stores key-value pairs and is not a subtype of Collection.

5. What is the difference between List, Set, and Queue?

Answer: List is ordered and allows duplicates. **Set** prevents duplicates. **Queue** is designed to process elements in an order, commonly FIFO.

6. What is the difference between ArrayList and LinkedList?

Answer: ArrayList uses a resizable array: fast indexed access, slower middle insertions/removals. **LinkedList** uses nodes: efficient additions/removals when positioned, slower indexed access.

7. What are the important features of ArrayList?

Answer: It preserves insertion order, allows duplicates and nulls, resizes automatically, and supports fast access by index. It is not synchronized by default.

8. How do you add elements to an ArrayList?

Answer: Use `list.add("Java")`. It appends the element and returns `true` if the list changes.

9. What is the difference between add() and add(index, element)?

Answer: `add(element)` appends. `add(index, element)` inserts at that position and shifts later elements right.

10. How do you access an element from an ArrayList?

Answer: Use `list.get(index)`, for example `list.get(0)`. Indexing starts at 0.

11. How do you remove an element from an ArrayList?

Answer: Use `remove(index)` to remove by position or `remove(object)` to remove the first matching value.

12. What is the difference between remove(int index) and remove(Object)?

Answer: `remove(int)` removes at an index and returns that element. `remove(Object)` removes the first equal object and returns a boolean.

13. How do you find the size of an ArrayList?

Answer: Call `list.size()`.

14. How do you check whether an ArrayList contains a particular element?

Answer: Call `list.contains(value)`. It uses `equals()` to compare objects.

15. How do you iterate through an ArrayList?

Answer: Use an enhanced for loop: `for (String s : list)`, an `Iterator`, or an indexed loop when indexes matter.

16. What is a Set?

Answer: A collection that contains no duplicate elements. Implementations include HashSet, LinkedHashSet, and TreeSet.

17. What is the main difference between List and Set?

Answer: A List allows duplicates and has positional access. A Set enforces uniqueness and generally has no index-based access.

18. Does a HashSet allow duplicate elements?

Answer: No. Adding an equal element again has no effect; the set keeps only one copy.

19. Does a HashSet maintain insertion order?

Answer: No. Its iteration order is not guaranteed and can change as the set changes.

20. What is the difference between HashSet and LinkedHashSet?

Answer: Both prevent duplicates. HashSet does not guarantee iteration order; LinkedHashSet preserves insertion order, with a small extra memory cost.

21. What is the difference between HashSet and TreeSet?

Answer: HashSet is typically faster for basic operations and has no sorted order. TreeSet keeps elements sorted, typically with O(log n) operations.

22. What is the purpose of TreeSet?

Answer: Use it when you need unique values automatically kept in sorted order, or need range operations such as `headSet()`.

23. How does TreeSet maintain its elements?

Answer: It uses a balanced tree (typically a Red-Black tree) and orders elements using natural ordering or a supplied Comparator.

24. What is a Queue?

Answer: A collection for holding elements before processing them. A common policy is FIFO: first in, first out.

25. What is the difference between add() and offer() in a Queue?

Answer: Both insert an element. `add()` throws an exception if insertion fails; `offer()` returns `false`.

26. What is the difference between poll() and remove() in a Queue?

Answer: Both remove and return the head. `poll()` returns `null` when empty; `remove()` throws an exception.

27. What is the difference between peek() and element()?

Answer: Both inspect the head without removing it. `peek()` returns `null` when empty; `element()` throws an exception.

28. What is a Map in Java?

Answer: An object that maps unique keys to values. Example: a student ID as a key and a student name as its value.

29. What is the difference between a Map and a Set?

Answer: A Set stores unique values only. A Map stores mappings: each unique key points to one value; values may repeat.

30. Does a HashMap allow duplicate keys?

Answer: No. Putting the same key again replaces its associated value.

31. Can a HashMap contain duplicate values?

Answer: Yes. Different keys may map to the same value.

32. What is the difference between HashMap and LinkedHashMap?

Answer: HashMap has no guaranteed order. LinkedHashMap preserves insertion order (or access order when configured).

33. What is the difference between HashMap and TreeMap?

Answer: HashMap is typically faster and unordered. TreeMap sorts entries by key, using natural ordering or a Comparator, with O(log n) operations.

34. What is the purpose of keySet(), values(), and entrySet()?

Answer: They return views of a map's keys, values, and key-value entries. `entrySet()` is often the best way to iterate over both keys and values.

35. What is an Iterator, and why is it used with collections?

Answer: An Iterator moves through a collection one element at a time using `hasNext()` and `next()`. Its `remove()` can safely remove the current item during iteration.

B. Strings in Java

36. What is a String in Java?

Answer: A String is an object representing a sequence of characters, such as `"Hello"`.

37. Why is String immutable in Java?

Answer: Once created, a String cannot change. This supports security, safe sharing, string pooling, and simpler behavior in concurrent code.

38. What is the difference between String and StringBuilder?

Answer: String is immutable, so repeated changes create new objects. StringBuilder is mutable and is more efficient for repeated concatenation or editing.

39. What is the difference between StringBuilder and StringBuffer?

Answer: Both are mutable. StringBuffer is synchronized and therefore thread-safe; StringBuilder is not synchronized and is usually faster in single-threaded code.

40. What is the difference between == and equals() when comparing Strings?

Answer: == checks whether two references point to the same object. `equals()` checks whether their character content is the same.

41. What is the String pool?

Answer: A special pool where Java reuses string literals and interned Strings. For example, identical literals can reference the same pooled object.

42. What is the use of the length() method?

Answer: `text.length()` returns the number of characters in the String.

43. What is the use of the charAt() method?

Answer: `text.charAt(index)` returns the character at a zero-based index.

44. What is the use of the substring() method?

Answer: It returns part of a String: `substring(begin)` goes to the end, while `substring(begin, end)` excludes the end index.

45. What is the difference between equals() and equalsIgnoreCase()?

Answer: `equals()` is case-sensitive. `equalsIgnoreCase()` ignores letter case when comparing.

46. What is the use of contains()?

Answer: `text.contains("Java")` returns true when the exact character sequence occurs in the String.

47. What is the use of indexOf() and lastIndexOf()?

Answer: `indexOf()` finds the first occurrence; `lastIndexOf()` finds the last. Both return -1 if absent.

48. What is the difference between replace() and replaceAll()?

Answer: `replace()` performs literal replacement. `replaceAll()` treats its first argument as a regular expression.

49. What is the use of trim()?

Answer: `trim()` returns a String with leading and trailing whitespace removed. For Unicode-aware stripping in newer Java, consider `strip()`.

50. How would you reverse a String in Java?

Answer: Use StringBuilder: `String reversed = new StringBuilder(text).reverse().toString();`